

发电负荷气象融合分析平台 软件概要设计说明书

版本：V0.1

数据分析课题组

文档修订记录

版本	*变化 状态	简要说明（变更内 容和变更范围）	日期	作者	参与者
V1.0.0	A	新建	2026-3-13	诸葛瑞乐、 孙志瑞、姜 林	诸葛瑞乐、孙志瑞、 姜林

*变化状态：A——增加，M——修改，D——删除，N——正式发布

文档审阅信息

版本	审阅人	角色	审阅日期	签字	备注
V1.0.0	诸葛瑞乐	项目负责人	2026.5.8	诸葛瑞乐	无

文档批准信息

版本	批准人	角色	批准日期	签字	备注
V1.0.0	诸葛瑞乐	项目负责人	2026.5.8	诸葛瑞乐	无

目录

1 引言	1
1.1 项目简要介绍	1
1.2 项目背景	1
1.3 项目的创新点	1
2 任务概述	2
2.1 目标	2
2.2 运行环境	2
2.3 功能需求	2
2.4 性能需求	3
3 总体设计	4
3.1 基本设计概念和处理流程	4
4 数据库设计	6
4.1 气象原始站点信息表: meta_weather_station	6
4.2 气象要素原始记录表: raw_weather_data	6
4.3 电厂测点元数据表: meta_dcs_point	7
4.4 电厂测点原始时序数据表: raw_dcs_data	7
4.5 融合宽表: fused_data_wide	7
4.6 模型管理表: model_registry	8
4.7 操作日志表: sys_oper_log	8
5 软件设计代码简述	10
5.1 数据预处理控制器 (PreprocessingController / main_preprocess.py)	10
5.2 特征工程与建模控制器 (AnalysisController / main_model.py)	13

1 引言

1.1 项目简要介绍

本项目旨在开发一个发电负荷气象融合分析平台，通过接入发电厂 DCS 运行数据和气象局气象观测预报数据，构建统一时间对齐的多源数据融合视图，实现发电负荷与供电需求预测、AGC 指令提前感知、高相关因素分析等核心功能，为发电运行优化和辅助调频决策提供数据支撑。

1.2 项目背景

本项目立足于发电企业在新能源高比例接入背景下，对负荷精准预测与灵活调节能力提升的实际需求。随着风电、光伏等新能源的大规模并网，电网负荷波动加剧，气象条件对火电机组承担的基础负荷影响日益显著。电厂 DCS 系统已积累海量运行测点数据，气象部门可提供全省实况和预报数据，但两类数据长期割裂。当前缺少一个集数据融合、特征分析、预测建模和可视化于一体的专业化分析平台。本项目将填补这一空白，通过对气象与机组运行数据的深度挖掘，提供可支撑运行决策的分析结论。

1.3 项目的创新点

1. 垂多源异构数据时间融合：首次将电厂秒级/分钟级 DCS 数据与气象小时级/日级数据在统一时间粒度下对齐，建立多维融合宽表，为后续分析打下基础。
2. 面向发电负荷影响的气象-设备联合特征工程：结合皮尔森相关系数、随机森林重要性和 mRMR 等方法，系统筛选对负荷影响最大的气象因素与机组运行参数，并运用 PCA 处理高维共线性。
3. AGC 指令提前感知模型：不局限于负荷预测，创新性地将 AGC 指令信号作为目标变量，以前置气象和机组状态信息构建预测分类/回归模型，探索辅助调频新路径。
4. 可配置的模块化分析流程：数据预处理策略、预测步长、模型选择等均采用参数化配置，支持不同机组、不同季节的灵活适配。

2 任务概述

2.1 目标

- 开发并部署“发电负荷气象融合分析平台”，实现以下目标：
- 完成气象数据、电厂 DCS 数据的统一接入、清洗、时间对齐和融合存储；
- 完成发电负荷及供电需求预测建模，支持 1h/4h/24h 多步长预测；
- 完成 AGC 信号提前感知的可行性分析与建模；
- 输出机组负荷高相关因素排序，并形成可视化分析结论。

2.2 运行环境

2.2.1 硬件环境

服务器	CPU: Intel Xeon 8 核 16 线程或同等算力，内存: 32GB，系统盘: 256GB SSD，数据盘: 2TB HDD
PC 电脑	建议 4 核 8 线程，16GB 及以上内存，支持 Chrome (90.0 及以上)、Edge (90.0 及以上) 浏览器

2.2.2 软件环境

此处详细说明软件运行的环境配置。

1. 服务器端操作系统: Linux (Ubuntu 22.04 LTS 或 CentOS 7)
2. Python 版本: 3.9+
3. 数据库: MySQL 8.4.x
4. Web 框架: FastAPI / Flask (接口服务)，前端可使用 Streamlit 轻量框架或 Vue+Echarts
5. 数据分析库: pandas、numpy、scikit-learn、statsmodels、tensorflow/ pytorch
6. 可视化库: matplotlib、seaborn、plotly、echarts

2.3 功能需求

数据接入与融合需求：

- a) 气象数据（全省实况、日照、基本预报、站点信息等）批量导入，自动解

析和校验。

- b) 电厂 DCS 测点数据（主蒸汽流量、风量、煤粉浓度、汽温汽压等百余个变量 CSV）批量导入，自动识别测点名称和时间列。
- c) 时间戳标准化与多源时间对齐，生成 15min（可配置）粒度的融合宽表。
- d) 缺失值自定义补全策略、异常值处理与日志记录。要性评估（方差选择、互信息、MIC 等）。

特征工程需求：

- a) 单变量相关性分析（皮尔森相关系数热力图）。
- b) 过滤法特征重 c) PCA 降维及随机森林重要性排序。
- d) 时间滞后特征与滑动窗口统计量构造。

分析建模与预测需求：

- a) 发电负荷预测模型（LSTM、XGBoost、多元回归，多种模型对比）。
- b) 供电需求预测模型（ARIMA、随机森林等）。
- c) AGC 信号提前感知模型（SVM 分类/SVR 回归）。
- d) 模型参数网格搜索优化、时间序列交叉验证、性能评估指标输出（MAE、MSE、 R^2 、AUC）。

可视化与结论需求：

- a) 负荷实测-预测对比折线图（误差带展示）。
- b) 高相关因素柱状图 Top-N。
- c) AGC 感知信号与实际指令同轴对比曲线。
- d) 一键导出数据分析结论报告（含图、表和文字结论）。

系统管理需求：

- a) 项目配置文件管理（测点映射、站点编码-名称映射）。
- b) 模型版本管理与训练历史记录。
- c) 操作日志与异常告警。

2.4 性能需求

2.4.1 数据精确度

- 温度数据精确到 1 位小数（℃）

-
- 压力数据精确到 2 位小数 (MPa)
 - 流量与负荷数据精确到 1 位小数 (t/h、MW)
 - 风速精确到 1 位小数 (m/s)
 - 相关系数精确到 4 位小数
 - 预测评估指标 (MAE、MSE) 精确到 4 位小数, MAPE 以百分比形式保留 2 位小数
 - 经费金额等涉及财务数据精确到分 (如适用)

2.4.2 时间特性要求

- 响应时间: 数据导入与融合生成宽表 (日级数据) ≤ 30 秒; 模型预测 (传入当前快照) ≤ 3 秒; 图表交互 ≤ 3 秒。
- 模型批量训练时间: 不设硬性上限, 日级更新允许 ≤ 2 小时; 支持离线/定时任务。
- 可视化大屏默认刷新频率: 1 分钟 (可配置)。

3 总体设计

3.1 基本设计概念和处理流程

本平台采用**数据层—分析层—应用层**的三层架构。

- 数据层负责双源数据接入和存储, 包含气象原始数据存储区、电厂 DCS 原始数据存储区、关系型数据库管理的融合宽表和模型文件存储。
- 分析层包含数据预处理引擎 (时间对齐、缺值异常处理)、特征工程引擎 (相关性分析、降维、升维)、建模引擎 (负荷预测、AGC 感知等模型训练与评估、参数优化)。
- 应用层提供 Web 前端交互界面和模型 REST API 服务, 前端控制请求流程, 后端处理数据和触发模型预测, 返回分析结果。

典型处理流程 (以负荷预测为例):

1. 用户上传气象 CSV 和电厂 DCS CSVs $\rightarrow$ 数据层接收并存储原始文件;
2. 预处理引擎进行格式校验、时间戳统一、15min 重采样、缺失值补全、异常值平滑 $\rightarrow$ 输出融合宽表存入数据库;

-
3. 特征工程引擎从宽表计算相关系数、执行 PCA 和时滞特征构造 → 输出特征集；
 4. 建模引擎加载特征集，按时间顺序划分训练集和测试集，训练 LSTM 负荷预测模型，用测试集评估 MAE 和 R^2 ，网格搜索调优超参数 → 保存最优模型文件；
 5. 用户在前端选择机组和时间窗口，点击“预测” → 应用层 API 调用模型推理，返回预测负荷曲线，前端以折线图展示并与实测值对比；
 6. 用户点击“导出结论” → 系统渲染图表和评估指标形成 PDF/Word 文档供下载。

4 数据库设计

4.1 气象原始站点信息表：meta_weather_station

表名：meta_weather_station

主键：station_code

字段	类型	说明
station_code	VARCHAR(10)	站点编码（主键）
station_name	VARCHAR(50)	站点名称
longitude	DECIMAL(7,4)	经度
latitude	DECIMAL(7,4)	纬度
altitude	DECIMAL(6,1)	海拔（m）
is_active	TINYINT	是否在用（1 在用/0 停用）

4.2 气象要素原始记录表：raw_weather_data

表名：raw_weather_data

主键：id（自增），索引：(station_code, obs_time)

字段	类型	说明
id	BIGINT	自增主键
station_code	VARCHAR(10)	站点编码
obs_time	DATETIME	观测时间
temperature	DECIMAL(4,1)	气温（℃）
pressure	DECIMAL(6,2)	气压（hPa）
humidity	DECIMAL(4,1)	相对湿度（%）
wind_speed	DECIMAL(4,1)	风速（m/s）
wind_dir	DECIMAL(5,1)	风向（°）
sunshine	DECIMAL(4,1)	日照时数（h）
precipitation	DECIMAL(5,1)	降水量（mm）

4.3 电厂测点元数据表：meta_dcs_point

表名：meta_dcs_point

主键：point_code

字段	类型	说明
point_code	VARCHAR(20)	测点编码（主键）
point_name	VARCHAR(100)	测点名称（如“主蒸汽流量 1”）
unit_id	VARCHAR(10)	所属机组编号
sensor_type	VARCHAR(20)	类型（流量/压力/温度/风量/煤量）
unit_range_min	DECIMAL(8,2)	量程下限
unit_range_max	DECIMAL(8,2)	量程上限
unit	VARCHAR(10)	单位（t/h, MPa, ℃等）

4.4 电厂测点原始时序数据表：raw_dcs_data

表名：raw_dcs_data

主键：id（自增），索引：(point_code, data_time)

字段	类型	说明
id	BIGINT	自增主键
point_code	VARCHAR(20)	测点编码
data_time	DATETIME(3)	采集时间（精确到毫秒）
value	DECIMAL(10,3)	采集值
quality_flag	TINYINT	质量标志（0 正常/1 可疑/2 异常）

4.5 融合宽表：fused_data_wide

表名：fused_data_wide

主键：id（自增），索引：(data_time, unit_id)

说明：以统一时间间隔（15min）为行，每行包含气象特征列和电厂测点特征列，是建模的直接输入。

字段	类型	说明
id	BIGINT	自增主键
data_time	DATETIME	统一时间戳
unit_id	VARCHAR(10)	机组编号

字段	类型	说明
气象特征...	—	温度、湿度、气压、风速风向等（每个站点对应电厂最近站点）
机组特征...	—	主蒸汽流量、汽包压力、风量、煤量、汽温等（15min 均值/最值）
target_load	DECIMAL(6,1)	机组负荷（MW）（分析建模目标变量）
agc_command	DECIMAL(5,1)	AGC 指令值（MW）（如有）
load_demand	DECIMAL(6,1)	供电需求（MW）（如有）

4.6 模型管理表：model_registry

表名：model_registry

主键：model_id

字段	类型	说明
model_id	INT	主键
model_name	VARCHAR(100)	模型名称（如 LSTM_Load_1h）
model_type	VARCHAR(30)	模型类型（LSTM/XGBoost/SVM）
target_variable	VARCHAR(50)	目标变量（load/agc）
train_start	DATE	训练数据起始日期
train_end	DATE	训练数据结束日期
metrics_json	TEXT	评估指标 JSON（MAE,R ² ）
file_path	VARCHAR(255)	模型文件存储路径
is_active	TINYINT	是否为当前在线模型
create_time	DATETIME	创建时间

4.7 操作日志表：sys_oper_log

表名：sys_oper_log

主键：oper_id

字段	类型	说明
oper_id	BIGINT	主键
oper_type	VARCHAR(30)	操作类型（数据导入/预处理/训练/预测/导出）
oper_detail	TEXT	操作详情
operator	VARCHAR(50)	操作人
oper_time	DATETIME	操作时间

字段	类型	说明
status	TINYINT	操作结果（0 失败/1 成功）
error_msg	TEXT	错误信息

5 软件设计代码简述

以 Python 后端核心代码为例，简要说明软件设计方法。

5.1 数据预处理控制器（PreprocessingController / main_preprocess.py）

模块路径：backend/preprocess/

```
# -*- coding: utf-8 -*-
"""
数据预处理主流程：时间对齐、缺失值处理、融合宽表生成
"""

import pandas as pd
import numpy as np
from datetime import timedelta

class DataFusionEngine:
    """
    气象-电厂数据融合引擎
    """

    def __init__(self, resample_interval='15min'):
        """
        初始化，设置统一重采样间隔
        :param resample_interval: 时间粒度，如'15min', '1H'
        """
        self.resample_interval = resample_interval

    def load_weather_data(self, csv_path: str) -> pd.DataFrame:
        """
        加载气象 CSV 数据，自动解析时间列，标准化列名
        :param csv_path: 气象 CSV 文件路径
        :return: DataFrame，包含标准化的 datetime 列和气象要素列
        """
        df = pd.read_csv(csv_path, encoding='utf-8')
        # 自动识别时间列（常见列名：obs_time, 观测时间, 时间, date_time）
        time_col_candidates = ['obs_time', '观测时间', '时间', 'date_time',
                               'Time']
        time_col = None
        for col in time_col_candidates:
            if col in df.columns:
                time_col = col
                break
```

```

        if time_col is None:
            raise ValueError("未找到时间列，请检查气象 CSV 文件表头")
        df['datetime'] = pd.to_datetime(df[time_col])
        df.set_index('datetime', inplace=True)
        # 重采样至统一间隔，数值列取均值
        df_resampled = df.resample(self.resample_interval).mean()
        return df_resampled

    def load_dcs_data(self, csv_paths: list) -> pd.DataFrame:
        """
        批量加载电厂 DCS 测点 CSV 文件，按时间索引合并为一张表
        :param csv_paths: DCS 各测点 CSV 文件路径列表
        :return: 合并后的宽表 DataFrame，每个测点一列
        """
        merged = None
        for path in csv_paths:
            # 根据文件名推断测点名称
            point_name = path.split('/')[-1].replace('.csv', '').replace('S',
        '')

            df = pd.read_csv(path, encoding='utf-8')
            # 自动识别时间列和数值列
            time_col = [c for c in df.columns if 'time' in c.lower() or '时
间' in c][0]
            val_col = [c for c in df.columns if c != time_col][0]
            df['datetime'] = pd.to_datetime(df[time_col])
            df = df[['datetime', val_col]].copy()
            df.rename(columns={val_col: point_name}, inplace=True)
            df.set_index('datetime', inplace=True)
            # 重采样至统一间隔
            df_resampled = df.resample(self.resample_interval).mean()
            if merged is None:
                merged = df_resampled
            else:
                merged = merged.join(df_resampled, how='outer')
        return merged

    def handle_missing_values(self, df: pd.DataFrame, strategy='ffill') ->
pd.DataFrame:
        """
        缺失值处理
        :param df: 输入 DataFrame
        :param strategy: 'ffill' 前向填充, 'bfill' 后向填充, 'mean' 均值, 'drop'
删除
        :return: 处理后的 DataFrame

```

```

        """
        if strategy == 'ffill':
            df = df.fillna(method='ffill').fillna(method='bfill')
        elif strategy == 'bfill':
            df = df.fillna(method='bfill').fillna(method='ffill')
        elif strategy == 'mean':
            df = df.fillna(df.mean())
        elif strategy == 'drop':
            df = df.dropna()
        return df

    def fuse(self, weather_csv: str, dcs_csv_list: list,
            missing_strategy='ffill', start_time=None, end_time=None) ->
pd.DataFrame:
        """
        核心融合方法：加载气象和 DCS 数据，统一时间对齐，处理缺值，生成融合宽
表
        :param weather_csv: 气象数据文件路径
        :param dcs_csv_list: 电厂 DCS 测点文件路径列表
        :param missing_strategy: 缺失值处理策略
        :param start_time: 可选，截取起始时间
        :param end_time: 可选，截取结束时间
        :return: 融合后的宽表 DataFrame
        """
        print("[INFO] 加载气象数据...")
        weather_df = self.load_weather_data(weather_csv)
        print("[INFO] 加载电厂 DCS 数据...")
        dcs_df = self.load_dcs_data(dcs_csv_list)
        print("[INFO] 合并双源数据...")
        fused = weather_df.join(dcs_df, how='inner') # 仅保留双源均有时间戳
的行
        print(f"[INFO] 融合后形状: {fused.shape}")
        print("[INFO] 处理缺失值...")
        fused = self.handle_missing_values(fused, strategy=missing_strategy)
        if start_time:
            fused = fused[fused.index >= pd.to_datetime(start_time)]
        if end_time:
            fused = fused[fused.index <= pd.to_datetime(end_time)]
        print(f"[INFO] 最终融合宽表形状: {fused.shape}")
        return fused

# 使用示例
if __name__ == '__main__':
    engine = DataFusionEngine(resample_interval='15min')

```

```

fused_df = engine.fuse(
    weather_csv='../data/weather/全省实况 201901.csv',
    dcs_csv_list=['../data/dcs/S 主蒸汽流量 1 265.csv',
                  '../data/dcs/S B 侧二次风量 166.csv'],
    missing_strategy='ffill'
)
fused_df.to_csv('../output/fused_data_wide.csv', encoding='utf-8-sig')
print("融合宽表已保存到 ../output/fused_data_wide.csv")

```

5.2 特征工程与建模控制器 (AnalysisController / main_model.py)

模块路径: backend/analysis/

```

# -*- coding: utf-8 -*-
"""
特征相关性分析与负荷预测建模
"""

import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import TimeSeriesSplit
from sklearn.metrics import mean_absolute_error, r2_score
import matplotlib.pyplot as plt
import seaborn as sns

class FeatureAnalyzer:
    """特征工程分析器"""

    @staticmethod
    def pearson_correlation_heatmap(df: pd.DataFrame, target_col: str,
top_k=20):
        """计算皮尔森相关系数并输出热力图"""
        corr = df.corr()[target_col].drop(target_col).sort_values(key=abs,
ascending=False)
        print(f"[Top-{top_k} 高相关因素]:")
        print(corr.head(top_k))
        top_features = corr.head(top_k).index.tolist()
        plt.figure(figsize=(12, 8))
        sns.heatmap(df[top_features + [target_col]].corr(), annot=True,
fmt='.3f', cmap='coolwarm')
        plt.title(f"Top-{top_k} Feature Correlation Heatmap")
        plt.tight_layout()
        plt.savefig('../output/correlation_heatmap.png', dpi=150)
        return corr

```



```

    @staticmethod
    def feature_importance_rf(df: pd.DataFrame, target_col: str, top_k=20):
        """随机森林特征重要性排序"""
        features = [c for c in df.columns if c != target_col]
        X = df[features].fillna(df[features].mean())
        y = df[target_col]
        model = RandomForestRegressor(n_estimators=100, random_state=42,
n_jobs=-1)
        model.fit(X, y)
        importances = pd.Series(model.feature_importances_,
index=features).sort_values(ascending=False)
        plt.figure(figsize=(10, 6))
        importances.head(top_k).plot(kind='barh')
        plt.title(f"Top-{top_k} Feature Importances (Random Forest)")
        plt.gca().invert_yaxis()
        plt.tight_layout()
        plt.savefig('../output/feature_importance.png', dpi=150)
        return importances

class LoadPredictor:
    """负荷预测器"""

    @staticmethod
    def create_lag_features(df, target_col, lag_steps=[1, 2, 4, 8, 16]):
        """构造时间滞后特征"""
        df_lag = df.copy()
        for lag in lag_steps:
            df_lag[f'{target_col}_lag_{lag}'] =
df_lag[target_col].shift(lag)
        return df_lag.dropna()

    @staticmethod
    def train_xgboost_predict(df, target_col, test_ratio=0.2):
        """XGBoost 训练与评估"""
        from xgboost import XGBRegressor
        features = [c for c in df.columns if c != target_col]
        split_idx = int(len(df) * (1 - test_ratio))
        train, test = df.iloc[:split_idx], df.iloc[split_idx:]
        X_train, y_train = train[features], train[target_col]
        X_test, y_test = test[features], test[target_col]
        model = XGBRegressor(n_estimators=200, learning_rate=0.05,
max_depth=6, random_state=42)
        model.fit(X_train, y_train)

```

```

        y_pred = model.predict(X_test)
        mae = mean_absolute_error(y_test, y_pred)
        r2 = r2_score(y_test, y_pred)
        print(f"XGBoost Test MAE: {mae:.4f}, R²: {r2:.4f}")
        # 绘制预测对比图
        plt.figure(figsize=(14, 5))
        plt.plot(test.index, y_test.values, label='Actual Load',
color='blue')
        plt.plot(test.index, y_pred, label='Predicted Load', color='red',
alpha=0.7)
        plt.fill_between(test.index, y_pred - mae, y_pred + mae, alpha=0.15,
color='red')
        plt.legend()
        plt.title(f"Load Prediction (MAE={mae:.2f} MW, R²={r2:.3f})")
        plt.tight_layout()
        plt.savefig('../output/load_prediction.png', dpi=150)
        return model, {'MAE': mae, 'R2': r2}

# 使用示例
if __name__ == '__main__':
    fused_df = pd.read_csv('../output/fused_data_wide.csv', index_col=0,
parse_dates=True)
    analyzer = FeatureAnalyzer()
    corr = analyzer.pearson_correlation_heatmap(fused_df,
target_col='target_load', top_k=20)
    fi = analyzer.feature_importance_rf(fused_df, target_col='target_load',
top_k=20)
    predictor = LoadPredictor()
    df_lag = predictor.create_lag_features(fused_df, 'target_load')
    model, metrics = predictor.train_xgboost_predict(df_lag, 'target_load')
    print("分析完成，图表已保存。")

```